Gradient Methods

Morten Hjorth-Jensen Email morten.hjorth-jensen@fys.uio.no[1]

Department of Physics and Center fo Computing in Science Education, University of Oslo, Oslo, Norway[1]

March 6, 2026

© 1999-2026, Morten Hjorth-Jensen Email morten.hjorth-jensen@fys.uio.no. Released under CC Attribution-NonCommercial 4.0 license

Overview of week March 2-6

Gradient methods:

1. Semi-Newton methods (Broyden's algorithm, Broyden-Farberg-Goldberg-Shanno algorithm, and Powell's algorithm)
2. Steepest descent and conjugate gradient descent
3. Stochastic reconfiguration if we get time

Teaching Material, videos and written material

1. These lecture notes and those from last week concerning the implementation of the gradients. We will revisit the codes from last week
2. Video on the Conjugate Gradient methods
3. Recommended background literature, Convex Optimization by Boyd and Vandenberghe. Their lecture slides are very useful (warning, these are some 300 pages).

Brief reminder on Newton-Raphson's method

Let us quickly remind ourselves how we derive the above method. Perhaps the most celebrated of all one-dimensional root-finding routines is Newton's method, also called the Newton-Raphson method. This method requires the evaluation of both the function f and its derivative f' at arbitrary points. If you can only calculate the derivative numerically and/or your function is not of the smooth type, we normally discourage the use of this method.

The equations

The Newton-Raphson formula consists geometrically of extending the tangent line at a current point until it crosses zero, then setting the next guess to the abscissa of that zero-crossing. The mathematics behind this method is rather simple. Employing a Taylor expansion for x sufficiently close to the solution s, we have

$$f(s) = 0 = f(x) + (s - x)f'(x) + \frac{(s - x)^2}{2} f''(x) + \dots.$$

Small values

For small enough values of the function and for well-behaved functions, the terms beyond linear are unimportant, hence we obtain

$$f(x) + (s - x)f'(x) \approx 0,$$

yielding

$$s \approx x - \frac{f(x)}{f'(x)}.$$

Having in mind an iterative procedure, it is natural to start iterating with

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Simple geometric interpretation

The above is Newton-Raphson's method. It has a simple geometric interpretation, namely x_{n+1} is the point where the tangent from $(x_n, f(x_n))$ crosses the x-axis. Close to the solution, Newton-Raphson converges fast to the desired result. However, if we are far from a root, where the higher-order terms in the series are important, the Newton-Raphson formula can give grossly inaccurate results. For instance, the initial guess for the root might be so far from the true root as to let the search interval include a local maximum or minimum of the function. If an iteration places a trial guess near such a local extremum, so that the first derivative nearly vanishes, then Newton-Raphson may fail totally

Extending to more than one variable

Newton's method can be generalized to systems of several non-linear equations and variables. Consider the case with two equations

$$\begin{aligned} f_1(x_1, x_2) &= 0 \\ f_2(x_1, x_2) &= 0, \end{aligned}$$

which we Taylor expand to obtain

$$\begin{aligned} 0 = f_1(x_1 + h_1, x_2 + h_2) = & \quad f_1(x_1, x_2) + h_1 \partial f_1 / \partial x_1 + h_2 \partial f_1 / \partial x_2 + \ldots \\ 0 = f_2(x_1 + h_1, x_2 + h_2) = & \quad f_2(x_1, x_2) + h_1 \partial f_2 / \partial x_1 + h_2 \partial f_2 / \partial x_2 + \ldots \end{aligned} .$$

Jacobian

Defining the Jacobian matrix $\hat{J}$ we have

$$\hat{J} = \begin{pmatrix} \partial f_1/\partial x_1 & \partial f_1/\partial x_2 \\ \partial f_2/\partial x_1 & \partial f_2/\partial x_2 \end{pmatrix},$$

we can rephrase Newton's method as

$$\begin{pmatrix} x_1^{n+1} \\ x_2^{n+1} \end{pmatrix} = \begin{pmatrix} x_1^n \\ x_2^n \end{pmatrix} + \begin{pmatrix} h_1^n \\ h_2^n \end{pmatrix},$$

where we have defined

$$\begin{pmatrix} h_1^n \\ h_2^n \end{pmatrix} = -\hat{\jmath}^{-1} \begin{pmatrix} f_1(x_1^n, x_2^n) \\ f_2(x_1^n, x_2^n) \end{pmatrix}.$$

Inverse of the Jacobian

We need thus to compute the inverse of the Jacobian matrix and it is to understand that difficulties may arise in case $\hat{J}$ is nearly singular.
It is rather straightforward to extend the above scheme to systems of more than two non-linear equations. In our case, the Jacobian matrix is given by the Hessian that represents the second derivative of the energy.

Steepest descent

The basic idea of gradient descent is that a function $F(\mathrm{x})$, $\mathrm{x} \equiv (x_1, \cdots, x_n)$, decreases fastest if one goes from x in the direction of the negative gradient $-\nabla F(\mathrm{x})$.
It can be shown that if

$$\mathrm{x}_{k+1} = \mathrm{x}_k - \gamma_k \nabla F(\mathrm{x}_k),$$

with $\gamma_k > 0$.
For γ_k small enough, then $F(\mathrm{x}_{k+1}) \leq F(\mathrm{x}_k)$. This means that for a sufficiently small γ_k we are always moving towards smaller function values, i.e a minimum.

More on Steepest descent

The previous observation is the basis of the method of steepest descent, which is also referred to as just gradient descent (GD). One starts with an initial guess x_0 for a minimum of F and computes new approximations according to

$$\mathrm{x}_{k+1} = \mathrm{x}_k - \gamma_k \nabla F(\mathrm{x}_k), \quad k \geq 0.$$

The parameter γ_k is often referred to as the step length or the learning rate within the context of Machine Learning.

The ideal

Ideally the sequence $\{\mathrm{x}_k\}_{k=0}$ converges to a global minimum of the function F. In general we do not know if we are in a global or local minimum. In the special case when F is a convex function, all local minima are also global minima, so in this case gradient descent can converge to the global solution. The advantage of this scheme is that it is conceptually simple and straightforward to implement.
However the method in this form has some severe limitations:
In machine learing we are often faced with non-convex high dimensional cost functions with many local minima. Since GD is deterministic we will get stuck in a local minimum, if the method converges, unless we have a very good intial guess. This also implies that the scheme is sensitive to the chosen initial condition.
Note that the gradient is a function of $\mathrm{x} = (x_1, \cdots, x_n)$ which makes it expensive to compute numerically.

The sensitiveness of the gradient descent

The gradient descent method is sensitive to the choice of learning rate γ_k. This is due to the fact that we are only guaranteed that $F(\mathrm{x}_{k+1}) \leq F(\mathrm{x}_k)$ for sufficiently small γ_k. The problem is to determine an optimal learning rate. If the learning rate is chosen too small the method will take a long time to converge and if it is too large we can experience erratic behavior.
Many of these shortcomings can be alleviated by introducing randomness. One such method is that of Stochastic Gradient Descent (SGD), see below.

Convex function

Convex function: Let $X \subset \mathbb{R}^n$ be a convex set. Assume that the function $f : X \to \mathbb{R}$ is continuous, then f is said to be convex if

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$$

for all $x_1, x_2 \in X$ and for all $t \in [0, 1]$. If $\leq$ is replaced with a strict inequaltiy in the definition, we demand $x_1 \neq x_2$ and $t \in (0, 1)$ then f is said to be strictly convex. For a single variable function, convexity means that if you draw a straight line connecting $f(x_1)$ and $f(x_2)$, the value of the function on the interval $[x_1, x_2]$ is always below the line as illustrated below.

Conditions on convex functions

In the following we state first and second-order conditions which ensures convexity of a function f. We write D_f to denote the domain of f, i.e the subset of R^n where f is defined. For more details and proofs we refer to: S. Boyd and L. Vandenberghe. Convex Optimization. Cambridge University Press.

First order condition

Suppose f is differentiable (i.e $\nabla f(x)$ is well defined for all x in the domain of f). Then f is convex if and only if D_f is a convex set and

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

holds for all $x, y \in D_f$. This condition means that for a convex function the first order Taylor expansion (right hand side above) at any point a global under estimator of the function. To convince yourself you can make a drawing of $f(x) = x^2 + 1$ and draw the tangent line to $f(x)$ and note that it is always below the graph.

Second condition

Second order condition

Assume that f is twice differentiable, i.e the Hessian matrix exists at each point in D_f. Then f is convex if and only if D_f is a convex set and its Hessian is positive semi-definite for all $x \in D_f$. For a single-variable function this reduces to $f''(x) \geq 0$. Geometrically this means that f has nonnegative curvature everywhere.

This condition is particularly useful since it gives us an procedure for determining if the function under consideration is convex, apart from using the definition.

More on convex functions

The next result is of great importance to us and the reason why we are going on about convex functions. In machine learning we frequently have to minimize a loss/cost function in order to find the best parameters for the model we are considering.
Ideally we want the global minimum (for high-dimensional models it is hard to know if we have local or global minimum). However, if the cost/loss function is convex the following result provides invaluable information:

Any minimum is global for convex functions

Consider the problem of finding $x \in \mathbb{R}^n$ such that $f(x)$ is minimal, where f is convex and differentiable. Then, any point x^* that satisfies $\nabla f(x^*) = 0$ is a global minimum.

This result means that if we know that the cost/loss function is convex and we are able to find a minimum, we are guaranteed that it is a global minimum.

Some simple problems

1. Show that $f(x) = x^2$ is convex for $x \in \mathbb{R}$ using the definition of convexity. Hint: If you re-write the definition, f is convex if the following holds for all $x, y \in D_f$ and any $\lambda \in [0, 1]$ $\lambda f(x) + (1 - \lambda)f(y) - f(\lambda x + (1 - \lambda)y) \geq 0$.
2. Using the second order condition show that the following functions are convex on the specified domain.
   - $f(x) = e^x$ is convex for $x \in \mathbb{R}$.
   - $g(x) = -\ln(x)$ is convex for $x \in (0, \infty)$.
3. Let $f(x) = x^2$ and $g(x) = e^x$. Show that $f(g(x))$ and $g(f(x))$ is convex for $x \in \mathbb{R}$. Also show that if $f(x)$ is any convex function than $h(x) = e^{f(x)}$ is convex.
4. A norm is any function that satisfy the following properties
   - $f(\alpha x) = |\alpha| f(x)$ for all $\alpha \in \mathbb{R}$.
   - $f(x + y) \leq f(x) + f(y)$
   - $f(x) \leq 0$ for all $x \in \mathbb{R}^n$ with equality if and only if $x = 0$

Using the definition of convexity, try to show that a function satisfying the properties above is convex.

Newton's method for $F(x) = 0$

Given iterate x_k, linearize:

$$F(x_k + s) \approx F(x_k) + J_k s, \qquad J_k = \nabla F(x_k) \in \mathbb{R}^{n \times n}.$$

Newton step solves

$$J_k s_k = -F(x_k), \qquad x_{k+1} = x_k + s_k.$$

Why quasi-Newton?

Computing and factorizing J_k (or the Hessian in optimization) can be expensive. Quasi-Newton methods build *approximations* $B_k \approx J_k$ or $H_k \approx J_k^{-1}$ from secant information.

Secant condition (multi-dimensional)

Define step and residual differences:

$$s_k = x_{k+1} - x_k, \qquad y_k = F(x_{k+1}) - F(x_k).$$

For the true Jacobian,

$$y_k \approx J(x_k)\, s_k.$$

Quasi-Newton methods enforce the **secant equation**

$$\boxed{B_{k+1} s_k = y_k}$$

with B_{k+1} "close" to B_k in a chosen sense.

Broyden's method: problem statement

Goal: update $B_k \approx J_k$ (or $H_k \approx J_k^{-1}$) using **one** new secant equation, avoiding full Jacobian evaluations.
We will derive the **good Broyden** update:

$$\boxed{B_{k+1} = B_k + \frac{(y_k - B_k s_k) s_k^\top}{s_k^\top s_k}}$$

and its inverse form.
(This is the minimum-change update in Frobenius norm satisfying the secant equation.)

Derivation: minimum-change Jacobian update

We want B_{k+1} such that:

$$B_{k+1} s_k = y_k$$

and B_{k+1} is as close as possible to B_k.
Let $\Delta B = B_{k+1} - B_k$. Optimization problem:

$$\min_{\Delta B} \ \|\Delta B\|_F^2 \quad \text{s.t.} \quad (B_k + \Delta B) s_k = y_k.$$

Constraint becomes:

$$\Delta B \, s_k = r_k, \qquad r_k := y_k - B_k s_k.$$

Solve the constrained problem

We minimize $\|\Delta B\|_F^2 = \mathrm{Tr}(\Delta B^\top \Delta B)$ subject to $\Delta B s_k = r_k$.
Use Lagrangian with multiplier $\lambda \in \mathbb{R}^n$:

$$\mathcal{L}(\Delta B, \lambda) = \mathrm{Tr}(\Delta B^\top \Delta B) + 2\,\lambda^\top (\Delta B s_k - r_k).$$

Stationarity w.r.t. ΔB:

$$\frac{\partial \mathcal{L}}{\partial \Delta B} = 2\Delta B + 2\lambda s_k^\top = 0 \quad \Rightarrow \quad \Delta B = -\lambda s_k^\top.$$

Impose constraint

$$\Delta B s_k = -\lambda(s_k^\top s_k) = r_k \quad \Rightarrow \quad \lambda = -\frac{r_k}{s_k^\top s_k}.$$

Thus

$$\boxed{\Delta B = \frac{r_k s_k^\top}{s_k^\top s_k}}$$

and therefore

$$\boxed{B_{k+1} = B_k + \frac{(y_k - B_k s_k) s_k^\top}{s_k^\top s_k}.}$$

Rank-one structure and interpretation

$$B_{k+1} = B_k + \frac{(y_k - B_k s_k) s_k^\top}{s_k^\top s_k}.$$

- Update is **rank-one** (outer product).
- Corrects B_k only along direction s_k to satisfy secant equation.
- Minimum change in Frobenius norm: most conservative correction consistent with new data.

Practical note

If $s_k^\top s_k$ is tiny, the step is unreliable; damping/regularization or restarting is used.

Broyden step computation

Given B_k, we compute a quasi-Newton step by solving:

$$B_k\, p_k = -F(x_k), \qquad x_{k+1} = x_k + p_k.$$

If solving linear systems is costly, update an inverse approximation $H_k \approx B_k^{-1}$.

Inverse (good) Broyden update

Enforce inverse secant:

$$H_{k+1} y_k = s_k,$$

and apply a minimum-change update in Frobenius norm:

$$\min_{\Delta H} \ \|\Delta H\|_F^2 \quad \text{s.t.} \quad (H_k + \Delta H) y_k = s_k.$$

Let $q_k := s_k - H_k y_k$. Then $\Delta H y_k = q_k$. As before, solution is rank-one:

$$\boxed{H_{k+1} = H_k + \frac{(s_k - H_k y_k)\, y_k^\top}{y_k^\top y_k}.}$$

Inverse (good) Broyden update

Algorithmic use

Step is $p_k = -H_k F(x_k)$ (matrix-vector product only).

Good vs bad Broyden (brief remark)

There is another rank-one update (**bad Broyden**) that minimizes a different measure (e.g. the change in the inverse, or weighted norms). In practice:

- **Good** Broyden is often preferred for robustness in Jacobian updates.
- Powell damping and trust-region strategies can stabilize both variants.

Optimization: Newton and quasi-Newton

For minimizing $f(x)$:

$$\nabla f(x^\star) = 0.$$

Newton step:

$$\nabla^2 f(x_k) p_k = -\nabla f(x_k).$$

Quasi-Newton methods build $B_k \approx \nabla^2 f(x_k)$ or $H_k \approx (\nabla^2 f(x_k))^{-1}$.
Define:

$$s_k = x_{k+1} - x_k, \qquad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

Secant condition (for Hessian approximation):

$$\boxed{B_{k+1} s_k = y_k.}$$

Why BFGS?

In optimization we want:

- symmetry: $B_{k+1} = B_{k+1}^\top$ (Hessian is symmetric),
- positive definiteness: $B_{k+1} \succ 0$ to guarantee descent directions,
- secant condition: $B_{k+1} s_k = y_k$.

BFGS is the most widely used quasi-Newton update satisfying these properties under mild conditions.

Curvature condition

Assume

$$\boxed{s_k^\top y_k > 0}$$

(curvature condition). It holds automatically if:

- f is strongly convex in a neighborhood and x_{k+1} is obtained by an appropriate line search,
- in practice, Wolfe conditions enforce it.

This condition ensures that BFGS preserves positive definiteness.

Derivation strategy for BFGS

We want an update $B_{k+1} = B_k + \Delta B$ that:

- is symmetric,
- satisfies secant: $(B_k + \Delta B)s = y$,
- is **closest** to B_k in a suitable metric.

A standard derivation constructs the inverse-Hessian update first (simpler), then inverts it to get B_{k+1}.

BFGS inverse update: statement

Let $H_k \approx (\nabla^2 f(x_k))^{-1}$ symmetric positive definite. Define $\rho_k := 1/(y_k^\top s_k)$. Then BFGS inverse update is:

$$\boxed{H_{k+1} = \left(I - \rho_k s_k y_k^\top\right) H_k \left(I - \rho_k y_k s_k^\top\right) + \rho_k s_k s_k^\top.}$$

- H_{k+1} is symmetric.
- If $H_k \succ 0$ and $y_k^\top s_k > 0$, then $H_{k+1} \succ 0$.
- It satisfies the inverse secant equation $H_{k+1} y_k = s_k$.

Proof: inverse secant property

We show $H_{k+1}y = s$ (drop k subscripts). Let $\rho = 1/(y^\top s)$ and define $V = I - \rho s y^\top$. Then

$$H_+ = VHV^\top + \rho s s^\top.$$

Compute $H_+ y$:

$$H_+ y = VHV^\top y + \rho s(s^\top y).$$

Since $V^\top y = (I - \rho y s^\top)y = y - \rho y(s^\top y) = y - y = 0$, we get $VHV^\top y = 0$. Also $\rho s(s^\top y) = \rho s(y^\top s) = s$. Thus $H_+ y = s$.

Proof: symmetry and positive definiteness

Symmetry is immediate from

$$H_+ = VHV^\top + \rho ss^\top$$

since both terms are symmetric if H is symmetric.
For positive definiteness, for any nonzero z:

$$z^\top H_+ z = (V^\top z)^\top H (V^\top z) + \rho (s^\top z)^2.$$

If $H \succ 0$ and $\rho > 0$ (i.e. $y^\top s > 0$), then both terms are ≥ 0 and not both zero unless $z = 0$. Hence $H_+ \succ 0$.

BFGS Hessian update (direct form)

Inverting the inverse update yields the standard BFGS update for $B_k \approx \nabla^2 f(x_k)$:

$$\boxed{B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k}.}$$

- Symmetric by construction.
- Preserves positive definiteness if $B_k \succ 0$ and $y_k^\top s_k > 0$.
- Satisfies $B_{k+1} s_k = y_k$.

How BFGS relates to Broyden family

The Broyden family (for optimization) is a one-parameter set of secant updates interpolating between rank-one (Broyden) and rank-two (DFP/BFGS) updates.

- BFGS is a particular member chosen for strong numerical behavior.
- In practice, BFGS (and limited-memory L-BFGS) is the default workhorse

in large-scale optimization in physics and ML.

Broyden (for $F(x) = 0$): algorithm sketch

Input: x_0, B_0 (or H_0), tolerance ε.
For $k = 0, 1, \ldots$:

1. Solve $B_k p_k = -F(x_k)$ (or $p_k = -H_k F(x_k)$).
2. Set $x_{k+1} = x_k + p_k$.
3. Compute $s_k = x_{k+1} - x_k$, $y_k = F(x_{k+1}) - F(x_k)$.
4. Update $B_{k+1} = B_k + \frac{(y_k - B_k s_k) s_k^\top}{s_k^\top s_k}$

(or $H_{k+1} = H_k + \frac{(s_k - H_k y_k) y_k^\top}{y_k^\top y_k}$).
Stop if $\|F(x_k)\| \leq \varepsilon$.

BFGS (for minimizing f): algorithm sketch

Input: x_0, $H_0 \succ 0$.
For $k = 0, 1, \ldots$:

1. Search direction: $p_k = -H_k \nabla f(x_k)$.
2. Line search: choose α_k satisfying Wolfe conditions.
3. Update iterate: $x_{k+1} = x_k + \alpha_k p_k$.
4. Set $s_k = x_{k+1} - x_k$, $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$.
5. Update H_{k+1} via BFGS inverse formula.

Stop if $\|\nabla f(x_k)\| \leq \varepsilon$.

Short summary

- Broyden: secant + minimum-change update $\Rightarrow$ rank-one Jacobian (or inverse) approximation.
- BFGS: symmetric, positive definite secant update $\Rightarrow$ robust optimization workhorse.
- Both replace expensive exact Jacobians/Hessians by learning curvature from iterates.

Takeaway

Quasi-Newton methods build an effective linear response model from observations.

Powell's method: Why Derivative-Free Methods?

Gradients may be unavailable or expensive:

- Monte Carlo noise
- Complex simulations
- Black-box models

Derivative-free methods include:

- Nelder-Mead simplex
- Powell's method
- Pattern search

Powell's method is particularly efficient for smooth functions.

Basic ideas: Concept of Direction Set Methods

Goal: minimize $f(\mathrm{x})$. How? Iteratively perform line minimizations along chosen directions.
Starting point

$$\mathrm{x}_0$$

Move along direction

$$\mathrm{d}$$

by minimizing

$$\phi(\alpha) = f(\mathrm{x}_0 + \alpha \mathrm{d})$$

This reduces a multidimensional problem to a sequence of 1D problems.

Line Minimization

For a given direction d:

$$\phi(\alpha) = f(\mathsf{x} + \alpha \mathsf{d})$$

Goal:

$$\alpha^* = \arg\min_{\alpha} \phi(\alpha)$$

Methods

- Golden section search
- Brent's method

Update

$$\mathsf{x}_{new} = \mathsf{x} + \alpha^* \mathsf{d}$$

Direction Set

Powell's method maintains a set of directions

$$\{\mathrm{d}_1, \mathrm{d}_2, \ldots, \mathrm{d}_n\}$$

Initially chosen as unit vectors:

$$\mathrm{d}_1 = (1, 0, 0, \ldots)$$

$$\mathrm{d}_2 = (0, 1, 0, \ldots)$$

etc.

One Iteration

Starting at point x_0.
Perform line minimizations sequentially:

$$\mathsf{x}_1 = \mathsf{x}_0 + \alpha_1 \mathsf{d}_1$$

$$\mathsf{x}_2 = \mathsf{x}_1 + \alpha_2 \mathsf{d}_2 \ldots$$

$$\mathsf{x}_n = \mathsf{x}_{n-1} + \alpha_n \mathsf{d}_n$$

Each step minimizes along a direction.

New Direction

After completing the cycle define

$$\mathrm{d}_{new} = \mathrm{x}_n - \mathrm{x}_0$$

This direction approximates the direction of largest improvement. Replace one of the existing directions with d_{new}.

Quadratic Functions

Consider quadratic function

$$f(\mathrm{x}) = \frac{1}{2}\mathrm{x}^T A\mathrm{x} + \mathrm{b}^T\mathrm{x} + c$$

For such functions

- conjugate directions allow exact minimization
- minimum found in n steps

Powell's method approximates conjugate directions without gradients.

Conjugate Directions

Two directions d_i and d_j are conjugate if

$$\mathrm{d}_i^T A \mathrm{d}_j = 0$$

For quadratic functions this ensures
successive minimizations do not undo previous ones.
Powell's algorithm attempts to generate such directions adaptively.

Powell Iteration

Given

$$\mathrm{x}_0$$

and directions

$$\mathrm{d}_1, \ldots, \mathrm{d}_n$$

1. Perform line minimizations along each direction
2. Record largest decrease in function
3. Construct new direction $\mathrm{d}_{new} = \mathrm{x}_n - \mathrm{x}_0$
4. Replace direction with largest decrease
5. Repeat

Stopping Criteria

Terminate when

- function change small $|f_{new} - f_{old}| < \epsilon$
- step size small $||\mathrm{x}_{new} - \mathrm{x}_{old}|| < \epsilon$
- maximum iterations reached

Advantages

- No derivatives required
- Efficient for moderate dimensions
- Works well for smooth functions

Limitations

- Not robust for noisy functions
- May fail for non-smooth problems
- Performance deteriorates in high dimensions

Short summary

- Powell's method is a derivative-free optimization algorithm
- Uses sequential line minimizations
- Updates search directions dynamically
- Approximates conjugate directions
- Efficient for moderate dimensional smooth problems

Standard steepest descent

Before we proceed, we would like to discuss the approach called the **standard Steepest descent**, which again leads to us having to be able to compute a matrix. It belongs to the class of Conjugate Gradient methods (CG).

The success of the CG method for finding solutions of non-linear problems is based on the theory of conjugate gradients for linear systems of equations. It belongs to the class of iterative methods for solving problems from linear algebra of the type

$$\hat{A}\hat{x} = \hat{b}.$$

In the iterative process we end up with a problem like

$$\hat{r} = \hat{b} - \hat{A}\hat{x},$$

where $\hat{r}$ is the so-called residual or error in the iterative process. When we have found the exact solution, $\hat{r} = 0$.

Gradient method

The residual is zero when we reach the minimum of the quadratic equation

$$P(\hat{x}) = \frac{1}{2}\hat{x}^T\hat{A}\hat{x} - \hat{x}^T\hat{b},$$

with the constraint that the matrix $\hat{A}$ is positive definite and symmetric. This defines also the Hessian and we want it to be positive definite.

Steepest descent method

We denote the initial guess for $\hat{x}$ as $\hat{x}_0$. We can assume without loss of generality that

$$\hat{x}_0 = 0,$$

or consider the system

$$\hat{A}\hat{z} = \hat{b} - \hat{A}\hat{x}_0,$$

instead.

Steepest descent method

One can show that the solution $\hat{x}$ is also the unique minimizer of the quadratic form

$$f(\hat{x}) = \frac{1}{2}\hat{x}^T \hat{A}\hat{x} - \hat{x}^T \hat{x}, \quad \hat{x} \in \mathrm{R}^n.$$

This suggests taking the first basis vector $\hat{r}_1$ (see below for definition) to be the gradient of f at $\hat{x} = \hat{x}_0$, which equals

$$\hat{A}\hat{x}_0 - \hat{b},$$

and $\hat{x}_0 = 0$ it is equal $-\hat{b}$.

Final expressions

We can compute the residual iteratively as

$$\hat{r}_{k+1} = \hat{b} - \hat{A}\hat{x}_{k+1},$$

which equals

$$\hat{b} - \hat{A}(\hat{x}_k + \alpha_k \hat{r}_k),$$

or

$$(\hat{b} - \hat{A}\hat{x}_k) - \alpha_k \hat{A}\hat{r}_k,$$

which gives

$$\alpha_k = \frac{\hat{r}_k^T \hat{r}_k}{\hat{r}_k^T \hat{A}\hat{r}_k}$$

leading to the iterative scheme

$$\hat{x}_{k+1} = \hat{x}_k - \alpha_k \hat{r}_k,$$

Conjugate gradient method

In the CG method we define so-called conjugate directions and two vectors $\hat{s}$ and $\hat{t}$ are said to be conjugate if

$$\hat{s}^T \hat{A} \hat{t} = 0.$$

The philosophy of the CG method is to perform searches in various conjugate directions of our vectors $\hat{x}_i$ obeying the above criterion, namely

$$\hat{x}_i^T \hat{A} \hat{x}_j = 0.$$

Two vectors are conjugate if they are orthogonal with respect to this inner product. Being conjugate is a symmetric relation: if $\hat{s}$ is conjugate to $\hat{t}$, then $\hat{t}$ is conjugate to $\hat{s}$.

Conjugate gradient method

An example is given by the eigenvectors of the matrix

$$\hat{v}_i^T \hat{A} \hat{v}_j = \lambda \hat{v}_i^T \hat{v}_j,$$

which is zero unless $i = j$.

Conjugate gradient method

Assume now that we have a symmetric positive-definite matrix $\hat{A}$ of size $n \times n$. At each iteration $i+1$ we obtain the conjugate direction of a vector

$$\hat{x}_{i+1} = \hat{x}_i + \alpha_i \hat{p}_i.$$

We assume that $\hat{p}_i$ is a sequence of n mutually conjugate directions. Then the $\hat{p}_i$ form a basis of R^n and we can expand the solution $\hat{A}\hat{x} = \hat{b}$ in this basis, namely

$$\hat{x} = \sum_{i=1}^{n} \alpha_i \hat{p}_i.$$

Conjugate gradient method

The coefficients are given by

$$\mathsf{Ax} = \sum_{i=1}^{n} \alpha_i \mathsf{Ap}_i = \mathsf{b}.$$

Multiplying with $\hat{p}_k^T$ from the left gives

$$\hat{p}_k^T \hat{A}\hat{x} = \sum_{i=1}^{n} \alpha_i \hat{p}_k^T \hat{A}\hat{p}_i = \hat{p}_k^T \hat{b},$$

and we can define the coefficients α_k as

$$\alpha_k = \frac{\hat{p}_k^T \hat{b}}{\hat{p}_k^T \hat{A}\hat{p}_k}$$

Conjugate gradient method and iterations

If we choose the conjugate vectors $\hat{p}_k$ carefully, then we may not need all of them to obtain a good approximation to the solution $\hat{x}$. We want to regard the conjugate gradient method as an iterative method. This will us to solve systems where n is so large that the direct method would take too much time.
We denote the initial guess for $\hat{x}$ as $\hat{x}_0$. We can assume without loss of generality that

$$\hat{x}_0 = 0,$$

or consider the system

$$\hat{A}\hat{z} = \hat{b} - \hat{A}\hat{x}_0,$$

instead.

Conjugate gradient method

One can show that the solution $\hat{x}$ is also the unique minimizer of the quadratic form

$$f(\hat{x}) = \frac{1}{2}\hat{x}^T \hat{A}\hat{x} - \hat{x}^T \hat{x}, \quad \hat{x} \in \mathrm{R}^n.$$

This suggests taking the first basis vector $\hat{p}_1$ to be the gradient of f at $\hat{x} = \hat{x}_0$, which equals

$$\hat{A}\hat{x}_0 - \hat{b},$$

and $\hat{x}_0 = 0$ it is equal $-\hat{b}$. The other vectors in the basis will be conjugate to the gradient, hence the name conjugate gradient method.

Conjugate gradient method

Let $\hat{r}_k$ be the residual at the k-th step:

$$\hat{r}_k = \hat{b} - \hat{A}\hat{x}_k.$$

Note that $\hat{r}_k$ is the negative gradient of f at $\hat{x} = \hat{x}_k$, so the gradient descent method would be to move in the direction $\hat{r}_k$. Here, we insist that the directions $\hat{p}_k$ are conjugate to each other, so we take the direction closest to the gradient $\hat{r}_k$ under the conjugacy constraint. This gives the following expression

$$\hat{p}_{k+1} = \hat{r}_k - \frac{\hat{p}_k^T \hat{A}\hat{r}_k}{\hat{p}_k^T \hat{A}\hat{p}_k}\hat{p}_k.$$

Conjugate gradient method

We can also compute the residual iteratively as

$$\hat{r}_{k+1} = \hat{b} - \hat{A}\hat{x}_{k+1},$$

which equals

$$\hat{b} - \hat{A}(\hat{x}_k + \alpha_k \hat{p}_k),$$

or

$$(\hat{b} - \hat{A}\hat{x}_k) - \alpha_k \hat{A}\hat{p}_k,$$

which gives

$$\hat{r}_{k+1} = \hat{r}_k - \hat{A}\hat{p}_k,$$

Using gradient descent methods, limitations

- **Gradient descent (GD) finds local minima of our function**. Since the GD algorithm is deterministic, if it converges, it will converge to a local minimum of our energy function. Because in ML we are often dealing with extremely rugged landscapes with many local minima, this can lead to poor performance.
- **GD is sensitive to initial conditions**. One consequence of the local nature of GD is that initial conditions matter. Depending on where one starts, one will end up at a different local minima. Therefore, it is very important to think about how one initializes the training process. This is true for GD as well as more complicated variants of GD.

Using gradient descent methods, limitations II

- **Gradients are computationally expensive to calculate for large datasets**. In many cases the energy function is a sum of terms, with one term for each data point. For example, in linear regression, $E \propto \sum_{i=1}^{n}(y_i - \mathrm{w}^T \cdot \mathrm{x}_i)^2$; for logistic regression, the square error is replaced by the cross entropy. To calculate the gradient we have to sum over *all* n data points. Doing this at every GD step becomes extremely computationally expensive. An ingenious solution to this, is to calculate the gradients using small subsets of the data called "mini batches". This has the added benefit of introducing stochasticity into our algorithm.

Using gradient descent methods, limitations III

- **GD is very sensitive to choices of learning rates**. GD is extremely sensitive to the choice of learning rates. If the learning rate is very small, the training process take an extremely long time. For larger learning rates, GD can diverge and give poor results. Furthermore, depending on what the local landscape looks like, we have to modify the learning rates to ensure convergence. Ideally, we would *adaptively* choose the learning rates to match the landscape.

Using gradient descent methods, limitations IV

- **GD treats all directions in parameter space uniformly.** Another major drawback of GD is that unlike Newton's method, the learning rate for GD is the same in all directions in parameter space. For this reason, the maximum learning rate is set by the behavior of the steepest direction and this can significantly slow down training. Ideally, we would like to take large steps in flat directions and small steps in steep directions. Since we are exploring rugged landscapes where curvatures change, this requires us to keep track of not only the gradient but second derivatives. The ideal scenario would be to calculate the Hessian but this proves to be too computationally expensive.

Using gradient descent methods, limitations V

- GD can take exponential time to escape saddle points, even with random initialization. As we mentioned, GD is extremely sensitive to initial condition since it determines the particular local minimum GD would eventually reach. However, even with a good initialization scheme, through the introduction of randomness, GD can still take exponential time to escape saddle points.

Stochastic reconfiguration. Quick reminders of calculational ingredients

Define local energy

$$E_L(x) = \frac{H\Psi(x)}{\Psi(x)}$$

Expectation value

$$E = \frac{\int dx |\Psi(x)|^2 E_L(x)}{\int dx |\Psi(x)|^2}$$

Monte Carlo sampling distribution

$$P(x) = \frac{|\Psi(x)|^2}{\langle \Psi | \Psi \rangle}$$

Metropolis Sampling

Generate samples

$$x_1, x_2, \ldots, x_M$$

using Metropolis algorithm.
Estimate expectation value

$$E \approx \frac{1}{M} \sum_{m=1}^{M} E_L(x_m)$$

Statistical error

$$\sigma_E \sim \frac{\sqrt{\mathrm{Var}(E_L)}}{\sqrt{M}}$$

Optimization Problem: Energy Gradient

Define logarithmic derivatives

$$O_k(x) = \frac{\partial \ln \Psi(x)}{\partial \alpha_k}$$

Energy gradient

$$\frac{\partial E}{\partial \alpha_k} = 2(\langle O_k E_L \rangle - \langle O_k \rangle \langle E_L \rangle)$$

Gradient Descent

Standard update

$$\alpha_k^{(n+1)} = \alpha_k^{(n)} - \eta \frac{\partial E}{\partial \alpha_k}$$

Problems:

- Parameter correlations
- Ill-conditioned landscape
- Large statistical noise

Better optimization required.

Imaginary-Time Evolution

Ground state projection

$$|\Psi(\tau)\rangle = e^{-\tau H}|\Psi(0)\rangle$$

For small step

$$|\Psi(\tau + \delta\tau)\rangle = (1 - \delta\tau H)|\Psi(\tau)\rangle$$

Idea: project evolution onto variational manifold.

Time Dependent Variational Principle (TDVP)

TDVP minimizes distance between

$$\partial_\tau |\Psi\rangle$$

and

$$-H|\Psi\rangle$$

within variational manifold.
Resulting equation

$$\sum_j S_{ij}\dot{\alpha}_j = -g_i$$

Covariance Matrix

Define

$$S_{ij} = \langle O_i O_j \rangle - \langle O_i \rangle \langle O_j \rangle$$

and

$$g_i = \langle O_i E_L \rangle - \langle O_i \rangle \langle E_L \rangle$$

Update rule

$$\delta\alpha = -\delta\tau S^{-1} g$$

Stochastic Reconfiguration (SR) Method

Stochastic Reconfiguration equation

$$\sum_j S_{ij} \delta \alpha_j = -\delta \tau g_i$$

Interpretation:

- Imaginary-time evolution
- Natural gradient descent
- TDVP projection

Geometric Interpretation

Wavefunctions form a manifold. Metric:

$$ds^2 = \sum_{ij} S_{ij} d\alpha_i d\alpha_j$$

This is the **quantum Fisher information metric**.

Quantum Fisher Information

For normalized wavefunction

$$|\psi(\boldsymbol{\alpha})\rangle$$

Fisher matrix

$$F_{ij} = 4(\langle O_i O_j \rangle - \langle O_i \rangle \langle O_j \rangle)$$

Thus

$$S_{ij} = F_{ij}/4$$

SR corresponds to natural gradient.

Ill Conditioning

The matrix S may become singular.
Regularization:

$$S_{ij} \to S_{ij} + \lambda \delta_{ij}$$

Typical

$$\lambda \sim 10^{-3}$$

SR Algorithm

1. Sample configurations
2. Compute $O_k(x)$
3. Compute local energies
4. Estimate S and g
5. Solve linear system
6. Update parameters